

PRACTICAL : WEEK 4

Session : 4

Task 1 :

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid[4];

    // Creating 4 child processes
    for (int i = 0; i < 4; i++)
    {
        pid[i] = fork();

        if (pid[i] == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());

            sleep(i + 1);

            printf("Child %d finished\n", i + 1);

            return 0;
        }
    }

    // Using wait()
    printf("\nUsing wait():\n");

    for (int i = 0; i < 2; i++)
    {
        wait(NULL);
        printf("Parent collected a child using wait()\n");
    }

    // Using waitpid()
    printf("\nUsing waitpid():\n");

    for (int i = 2; i < 4; i++)
    {
        waitpid(pid[i], NULL, 0);
        printf("Parent collected Child %d using waitpid()\n", i + 1);
    }

    printf("\nParent process finished.\n");

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week4
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ nano wait_waitpid.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ gcc wait_waitpid.c -o wait_waitpid
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ ./wait_waitpid
Child 1: PID = 529
Child 2: PID = 530
Child 3: PID = 531

Child 4: PID = 532
Using wait():
Child 1 finished
Parent collected a child using wait()
Child 2 finished
Parent collected a child using wait()

Using waitpid():
Child 3 finished
Parent collected Child 3 using waitpid()
Child 4 finished
Parent collected Child 4 using waitpid()

Parent process finished.
```

OBSERVATION :

The parent process successfully created four child processes using `fork()`. The child processes executed independently and terminated after completing their assigned execution time. The parent process used `wait()` to wait for any child process to terminate, while `waitpid()` was used to wait for specific child processes using their process IDs. The execution demonstrated that `wait()` provides general synchronization with child processes, whereas `waitpid()` provides more control by allowing the parent to synchronize with a particular child process. The parent process terminated only after all the child processes were properly collected.

Task 2 :

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

CODE:

Zombie process

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child process is running.\n");
        printf("Child PID: %d\n", getpid());

        return 0;
    }
    else
    {
        printf("Parent process is running.\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        sleep(30);

        printf("Parent process finished.\n");
    }

    return 0;
}
```

Eliminate Zombie process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child process is running.\n");
        printf("Child PID: %d\n", getpid());

        return 0;
    }
    else
    {
        printf("Parent process is running.\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Parent process finished.\n");
    }

    return 0;
}
```

OUTPUT :

Zombie Process

```
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ nano zombie.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ gcc zombie.c -o zombie
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ ./zombie
```

```
Parent process is running.
Parent PID: 692
Child PID: 693
Child process is running.
Child PID: 693
Parent process finished.
```

```
ananya@Ananya:~$ ps -el
F S      UID      PID      PPID      C  PRI      NI     ADDR     SZ  WCHAN    TTY          TIME CMD
4 S      0         1         0  0  80      0      -  6097    -    ?          ?         00:00:01 systemd
5 S      0         2         1  0  80      0      -   795    -    hvcb0     ?         00:00:00 init-systemd(Ub
0 S      0         7         2  0  80      0      -   795    -    hvcb0     ?         00:00:00 init
4 S      0        50         1  0  79     -1 - 12600    -    ?         ?         00:00:00 systemd-journal
4 S    989        82         1  0  80      0      -  5635    -    ?         ?         00:00:00 systemd-resolve
4 S      0        92         1  0  80      0      -  8818    -    ?         ?         00:00:00 systemd-udevd
4 S      0       111         1  0  80      0      -   722    -    ?         ?         00:00:00 chronyd-starter
4 S      0       112         1  0  80      0      -  1118    -    ?         ?         00:00:00 cron
4 S    995       113         1  0  80      0      -  2205    -    ?         ?         00:00:00 dbus-daemon
4 S      0       130         1  0  80      0      - 11023    -    ?         ?         00:00:00 networkd-dispat
4 S      0       156         1  0  80      0      -   4609    -    ?         ?         00:00:00 systemd-logind
4 S      0       161         1  0  80      0      - 448075   -    ?         ?         00:00:00 wsl-pro-service
4 S    100       183         1  0  80      0      - 55137    -    ?         ?         00:00:00 rsyslogd
4 S    988       220        111  0  80      0      -   5138    -    ?         ?         00:00:00 chronyd
1 S    988       245        220  0  80      0      -   3023    -    ?         ?         00:00:00 chronyd
4 S      0       265         1  0  80      0      -  1373    -    tty1      ?         00:00:00 agetty
4 S      0       266         1  0  80      0      - 30864    -    ?         ?         00:00:00 unattended-upgr
5 S      0       322         2  0  80      0      -   799    -    ?         ?         00:00:00 SessionLeader
5 S      0       323        322  0  80      0      -   799    -    ?         ?         00:00:00 Relay(324)
4 S   1000       324        323  0  80      0      -  1567 do_wai pts/0     ?         00:00:00 bash
4 S      0       325         2  0  80      0      -  2162    -    ?         ?         00:00:00 login
4 S   1000       377         1  0  80      0      -  5584    -    ?         ?         00:00:00 systemd
5 S   1000       379        377  0  80      0      -  5727    -    ?         ?         00:00:00 (sd-pam)
4 S   1000       405        325  0  80      0      -  1534 poll_s pts/1     ?         00:00:00 bash
5 S      0       586         2  0  80      0      -   796    -    ?         ?         00:00:00 SessionLeader
5 S      0       587        586  0  80      0      -   800    -    ?         ?         00:00:00 Relay(593)
4 S   1000       593        587  0  80      0      -  1567 do_wai pts/2     ?         00:00:00 bash
0 S   1000       692        324  0  80      0      -   692 hrtimr pts/0     ?         00:00:00 zombie
1 Z   1000       693        692  0  80      0      -     0    -    pts/0     ?         00:00:00 zombie
0 R   1000       699        593  0  80      0      -  1798    -    pts/2     ?         00:00:00 ps

ananya@Ananya:~$ ps -o pid,ppid,state,cmd -p 693
  PID   PPID  S  CMD
   693    692  Z  [zombie] <defunct>
ananya@Ananya:~$
```

Eliminate Zombie process

```
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ nano zombie.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ nano zombie.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ gcc zombie.c -o zombie
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ ./zombie
```

Parent process is running.

Parent PID: 771

Child PID: 772

Child process is running.

Child PID: 772

Parent process finished.

```
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week4$ |
```

```
ananya@Ananya:~$ ps -el
F S      UID      PID      PPID    C  PRI   NI   ADDR  SZ  WCHAN  TTY          TIME CMD
4 S      0         1         0  0  80    0   - 6097  -    ?           ?      00:00:01 systemd
5 S      0         2         1  0  80    0   - 795   -    hvc0        ?      00:00:00 init-systemd(Ub
0 S      0         7         2  0  80    0   - 795   -    hvc0        ?      00:00:00 init
4 S      0        50         1  0  79   -1  -12600 -    ?           ?      00:00:00 systemd-journal
4 S     989        82         1  0  80    0   - 5635 -    ?           ?      00:00:00 systemd-resolve
4 S      0        92         1  0  80    0   - 8818 -    ?           ?      00:00:00 systemd-udevd
4 S      0       111         1  0  80    0   - 722   -    ?           ?      00:00:00 chronyd-starter
4 S      0       112         1  0  80    0   - 1118 -    ?           ?      00:00:00 cron
4 S     995       113         1  0  80    0   - 2205 -    ?           ?      00:00:00 dbus-daemon
4 S      0       130         1  0  80    0   - 11023 -    ?           ?      00:00:00 networkd-dispat
4 S      0       156         1  0  80    0   - 4609 -    ?           ?      00:00:00 systemd-logind
4 S      0       161         1  0  80    0   - 448075 -    ?           ?      00:00:00 wsl-pro-service
4 S     100       183         1  0  80    0   - 55137 -    ?           ?      00:00:00 rsyslogd
4 S     988       220       111  0  80    0   - 5138 -    ?           ?      00:00:00 chronyd
1 S     988       245       220  0  80    0   - 3023 -    ?           ?      00:00:00 chronyd
4 S      0       265         1  0  80    0   - 1373 -    tty1        ?      00:00:00 agetty
4 S      0       266         1  0  80    0   - 30864 -    ?           ?      00:00:00 unattended-upgr
5 S      0       322         2  0  80    0   - 799   -    ?           ?      00:00:00 SessionLeader
5 S      0       323       322  0  80    0   - 799   -    ?           ?      00:00:00 Relay(324)
4 S    1000       324       323  0  80    0   - 1567 poll_s pts/0      ?      00:00:00 bash
4 S      0       325         2  0  80    0   - 2162 -    ?           ?      00:00:00 login
4 S    1000       377         1  0  80    0   - 5584 -    ?           ?      00:00:00 systemd
5 S    1000       379       377  0  80    0   - 5727 -    ?           ?      00:00:00 (sd-pam)
4 S    1000       405       325  0  80    0   - 1534 poll_s pts/1      ?      00:00:00 bash
5 S      0       586         2  0  80    0   - 796   -    ?           ?      00:00:00 SessionLeader
5 S      0       587       586  0  80    0   - 800   -    ?           ?      00:00:00 Relay(593)
4 S    1000       593       587  0  80    0   - 1567 do_wai pts/2      ?      00:00:00 bash
0 R    1000       782       593  0  80    0   - 1798 -    pts/2      ?      00:00:00 ps
```

```
ananya@Ananya:~$ ps -o pid,ppid,state,cmd -p 772
```

```
    PID    PPID S  CMD
```

```
ananya@Ananya:~$ █
```

OBSERVATION :

Initially, a child process was allowed to terminate while the parent process continued its execution without calling `wait()`. As a result, the terminated child remained in the process table as a zombie process. The process was investigated using the `ps` command, where it appeared with the process state `Z` and was displayed as `<defunct>`. The program was then modified to use `wait(NULL)` in the parent process. After synchronization was added, the parent collected the child's termination status, and the child process no longer remained in the process table as a zombie.